

Community Detection in Graphs

Thoai Nam

High Performance Computing Lab (HPC Lab)

Faculty of Computer Science and Technology

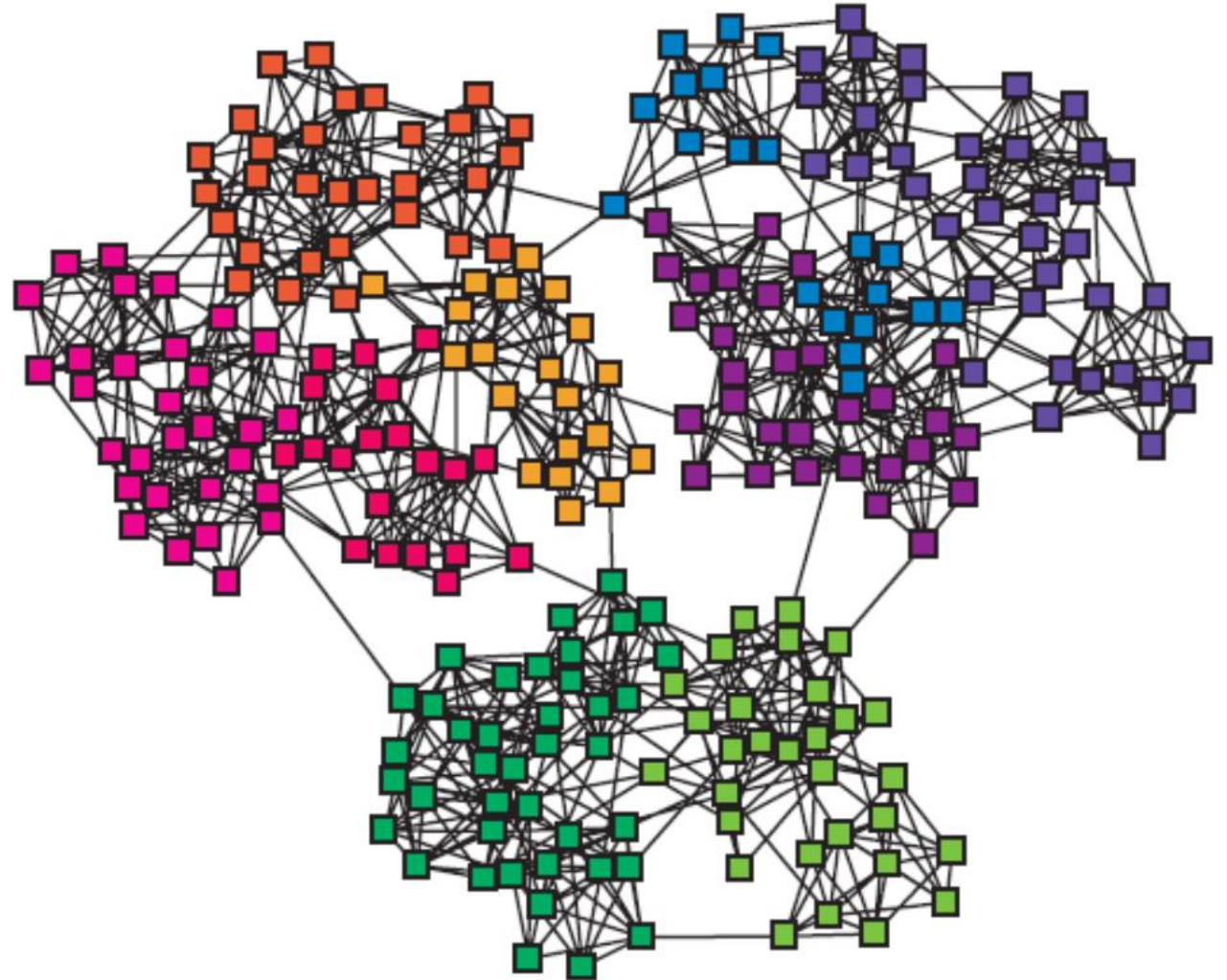
HCMC University of Technology

Topics

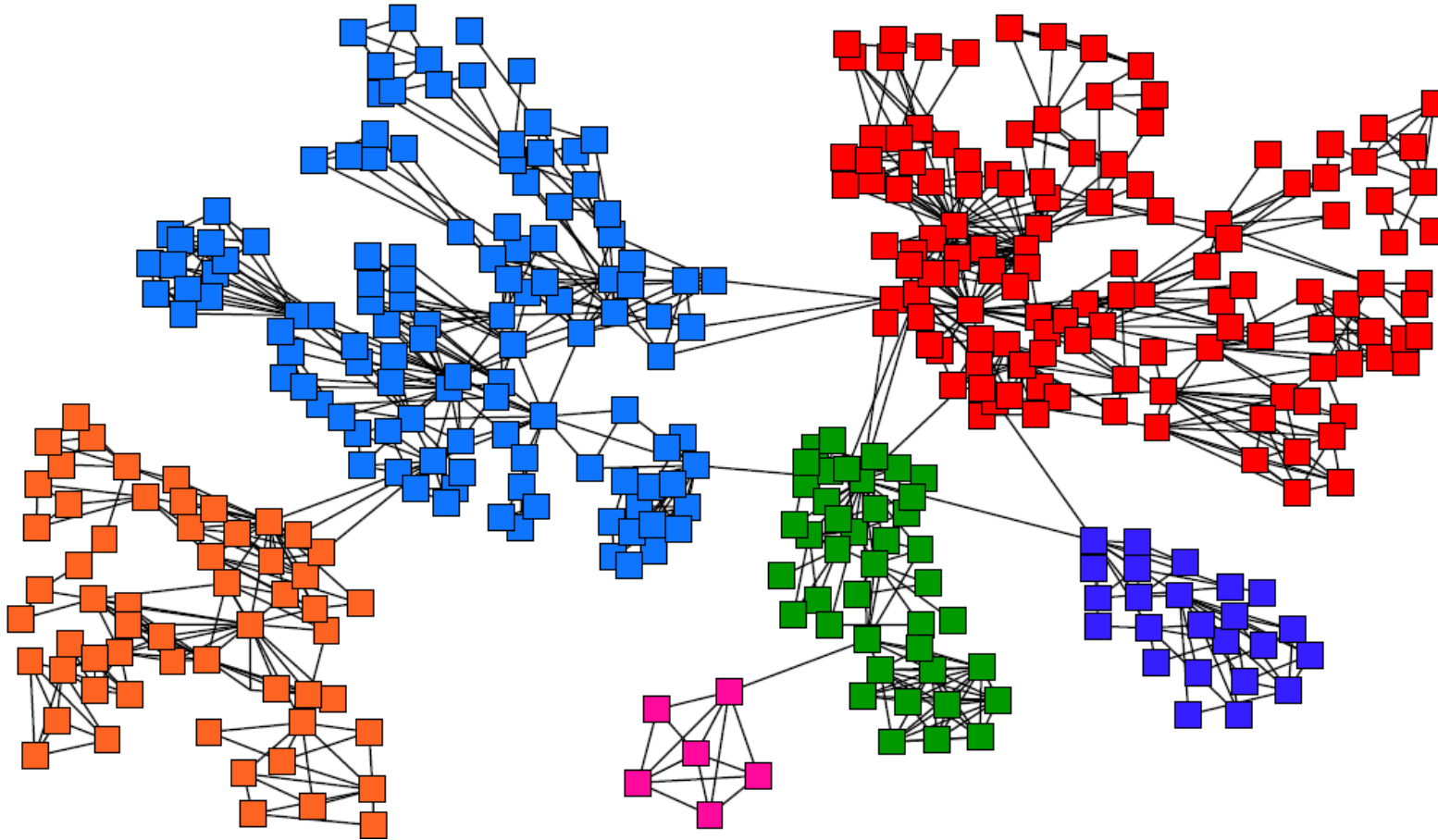
- PPR in community detection in graphs
- Modularity maximization
- k-NN graph construction

Networks & Communities

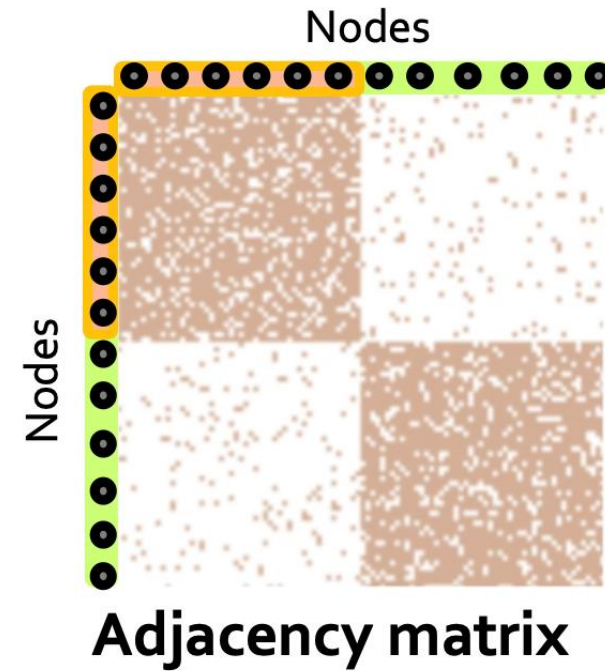
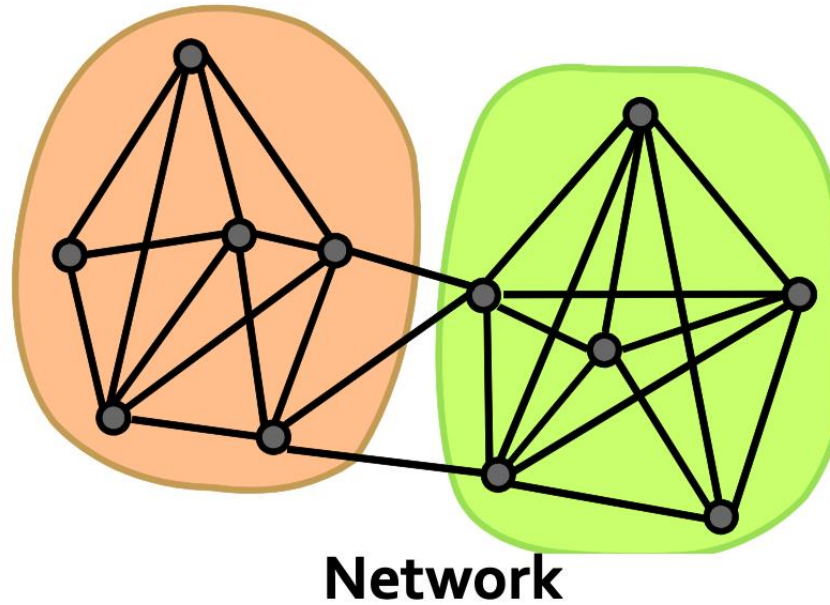
- We often think of networks being organized into **modules, cluster, communities**:



Goal: Find Densely Linked Clusters

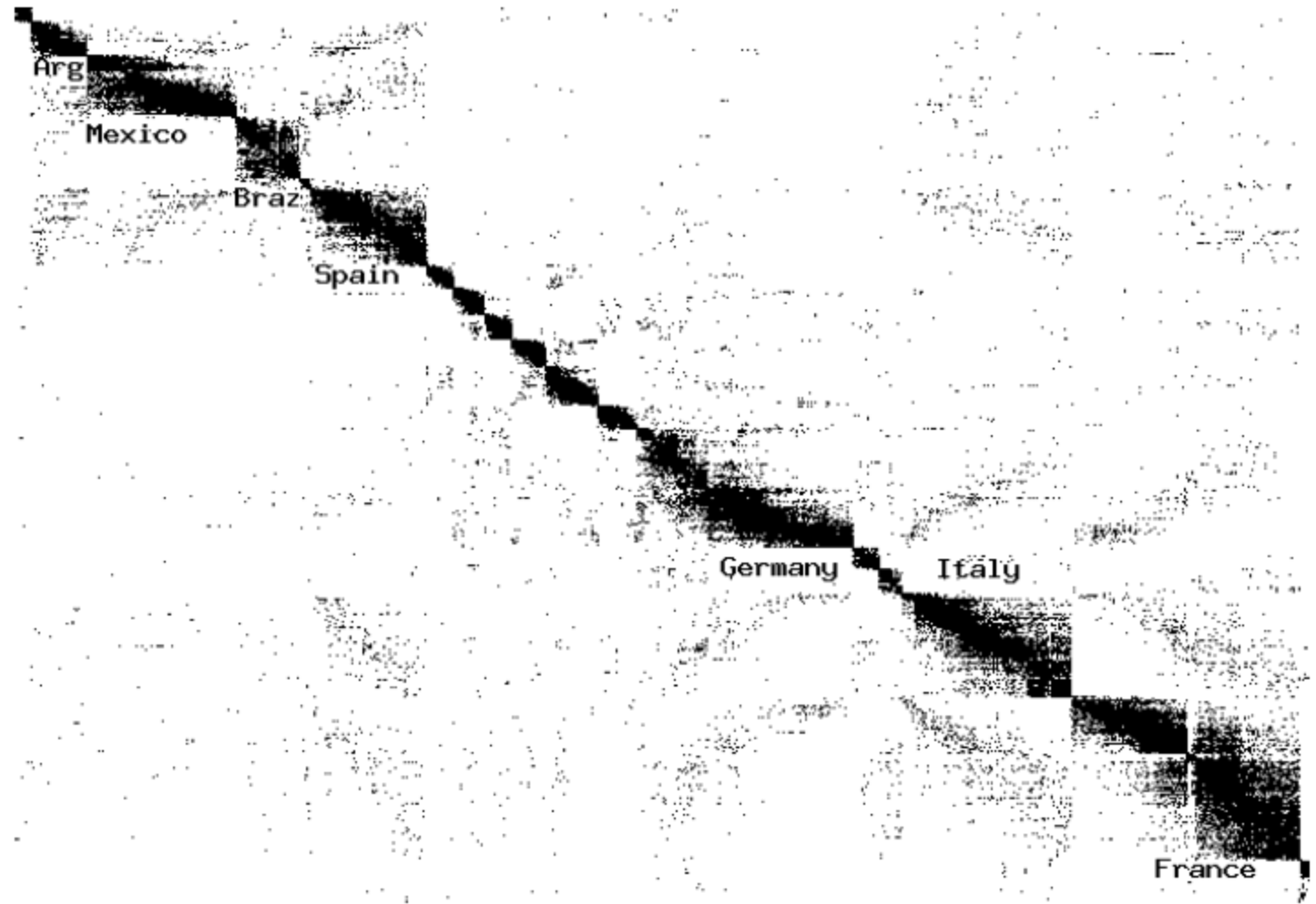


Non-overlapping clusters



Movies and Actors

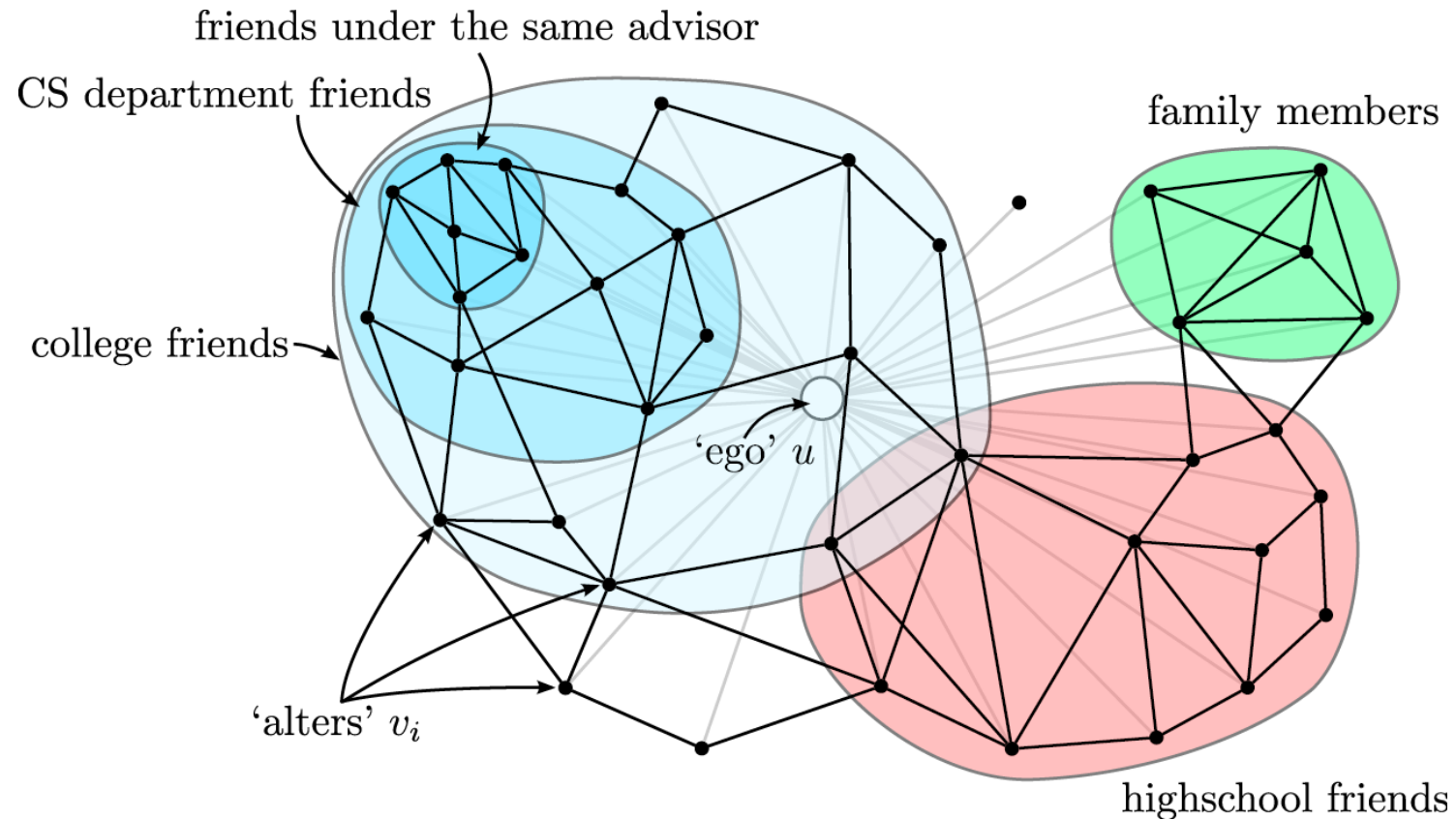
- Clusters in Movies-to-Actors graph:



[Andersen, Lang: Communities from seed sets, 2006]

Twitter & Facebook

- Discovering social circles, circles of trust:



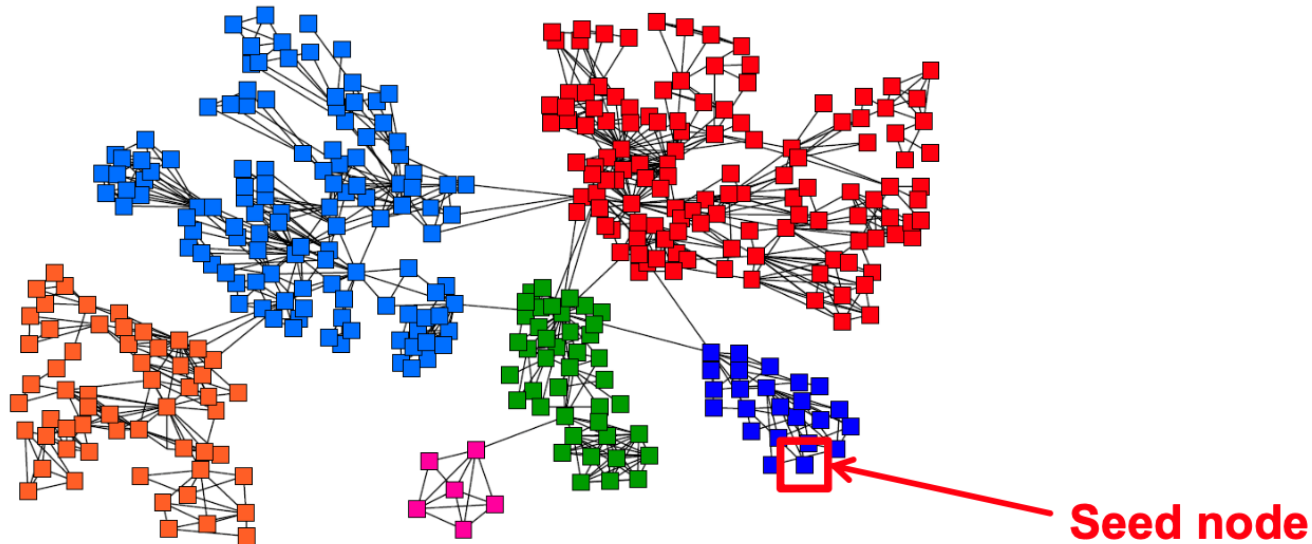
[McAuley, Leskovec: Discovering social circles in ego networks, 2012]

The setting

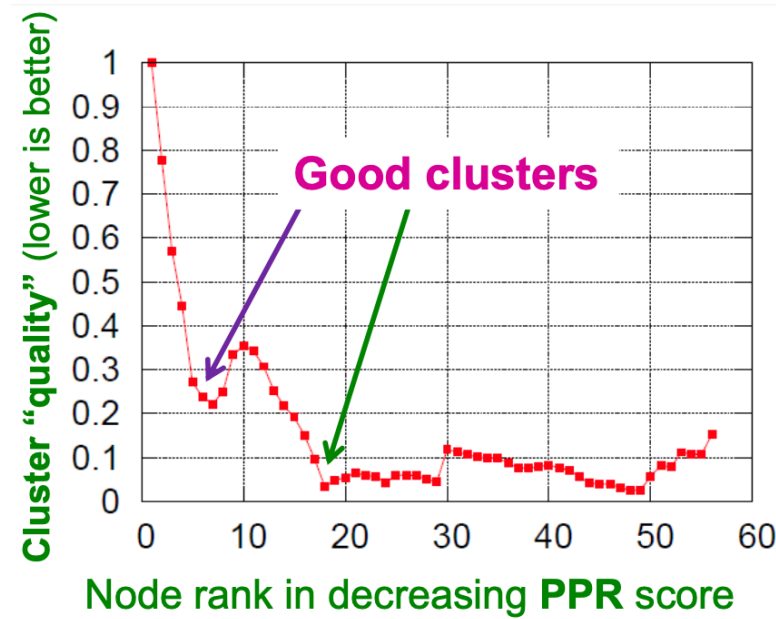
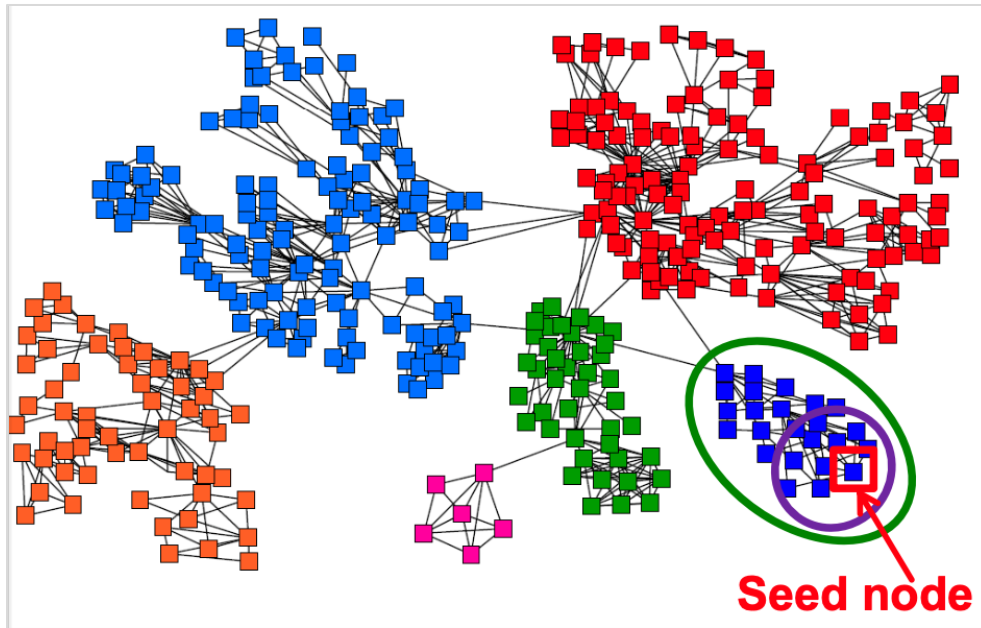
- **Graph is large**
 - **Assume the graph fits in main memory**
 - For example, to work with a 200M node and 2B edge graph one needs approx. 16GB RAM
 - But the graph is too big for running anything more than linear time algorithms
- **We will cover a PageRank based algorithm for finding dense clusters**
 - The runtime of the algorithm will be proportional to the cluster size (not the graph size!)

Idea: Seed nodes

- **Discovering clusters based on seed nodes**
 - **Given:** Seed node s
 - Compute (approximate) **Personalized PageRank (PPR)** around node s (teleport set= $\{s\}$)
 - Idea is that if s belongs to a nice cluster, the random walk will get **trapped** inside the cluster



Seed nodes: Intuition

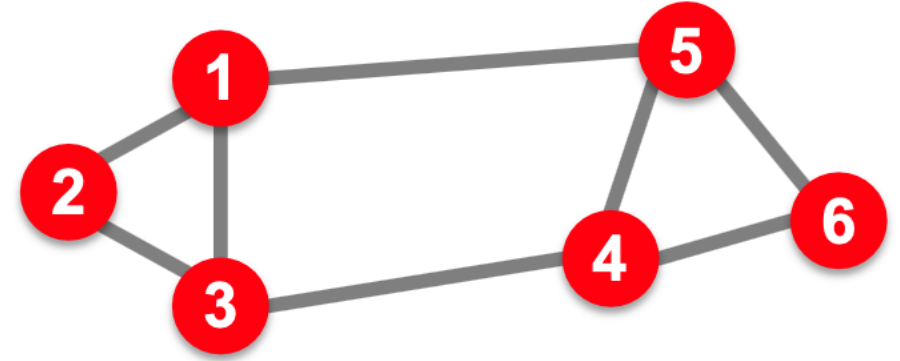


■ Algorithm outline:

- Pick a seed node s of interest
- Run **PPR** with teleport set = $\{s\}$
- Sort the nodes by the decreasing **PPR score**
- **Sweep** over the nodes and find **good clusters**

What makes a good cluster?

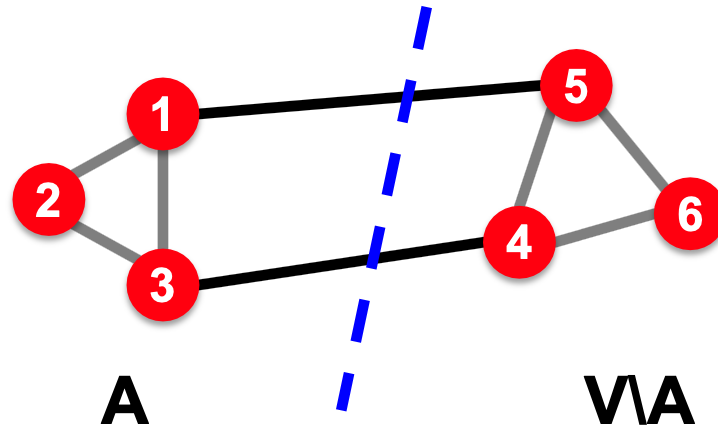
- Undirected graph $G(V, E)$:
- Bi-partitioning task:
Divide vertices into two disjoint groups A, B



- Questions:
How can we define a “good” partition of G ?
How can we efficiently identify such a partition?

What makes a good cluster?

- What makes a good cluster?
 - Maximize the number of within-cluster connections
 - Minimize the number of between-cluster connections

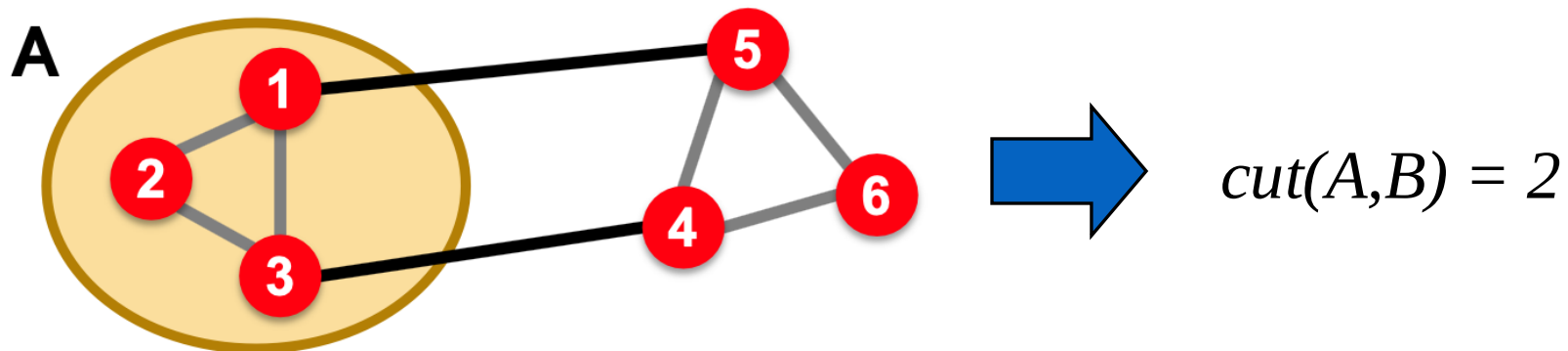


Graph cuts

- Express cluster quality as a function of the “edge cut” of the cluster
- Cut:** Set of edges (edge weights) with only one node in the cluster:

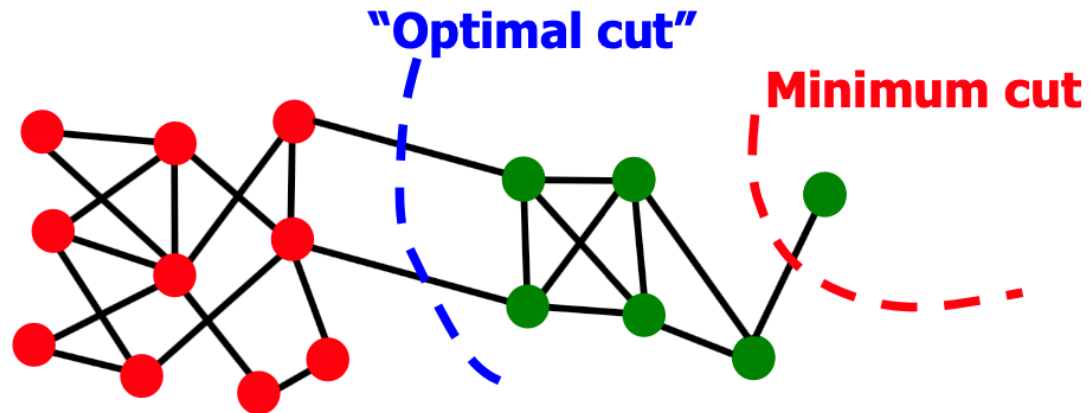
$$cut(A) = \sum_{i \in A, j \notin A} w_{ij}$$

Note: This works for weighted and unweighted (set all $w_{ij}=1$) graphs



Cut score

- **Partition quality: Cut score**
 - Quality of a cluster is the weight of connections pointing outside the cluster
- **Degenerate case:**



- **Problem:**
 - Only considers external cluster connections
 - Does not consider internal cluster connectivity

Graph partitioning criteria

■ Criterion: **Conductance**

Connectivity of the group to the rest of the network relative to the density of the group

$$\phi(A) = \frac{|\{(i, j) \in E; i \in A, j \notin A\}|}{\min(\text{vol}(A), 2m - \text{vol}(A))}$$

$\text{vol}(A)$: total weight of the edges with at least one endpoint in A : $\text{vol}(A) = \sum_{i \in A} k_i$

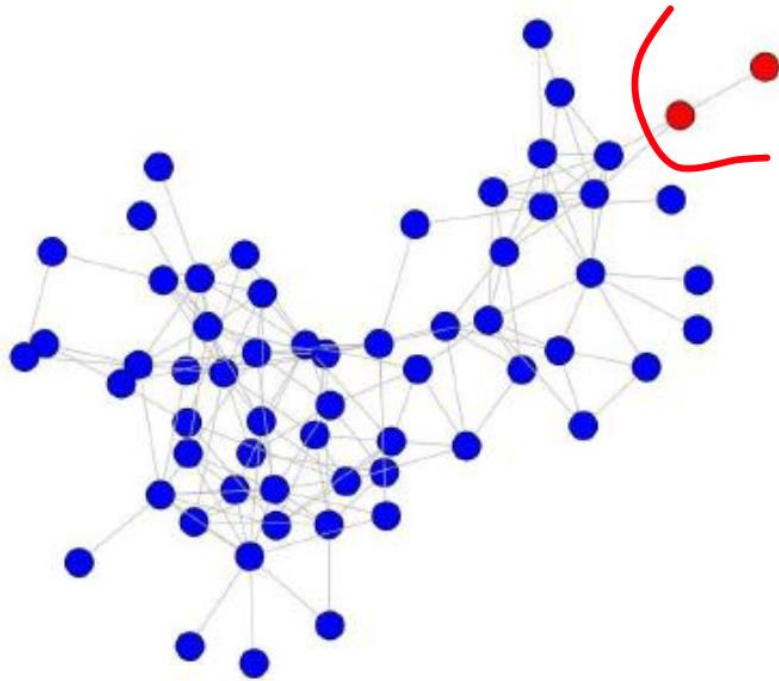
- $\text{Vol}(A) = 2 * \text{\#edges inside } A + \text{\#edges pointing out of } A$

m ... number of edges of the graph
 d_i ... degree of node i
 E ... edge set of the graph

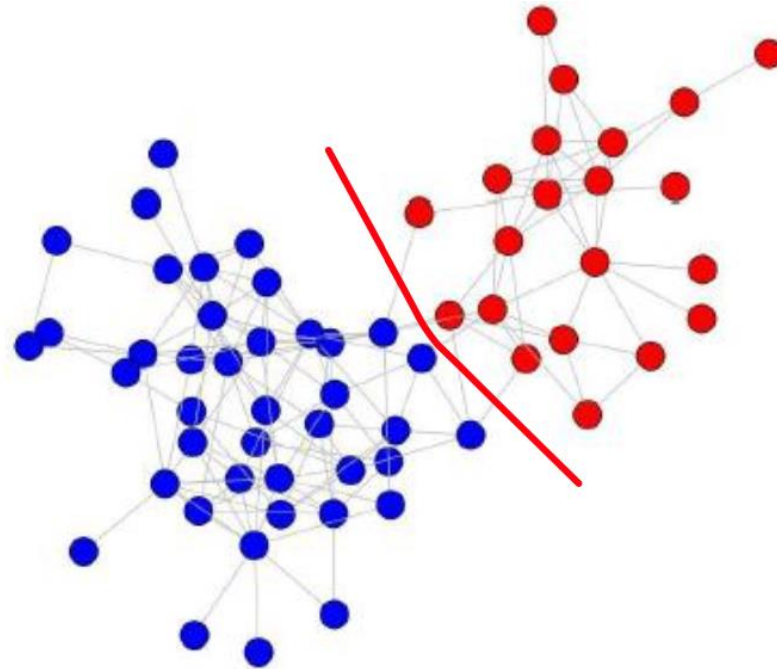
■ Why use this criterion?

- Produces more balanced partitions

Example: conductance score



$$\phi = 2/4 = 0.5$$



$$\phi = 6/92 = 0.065$$

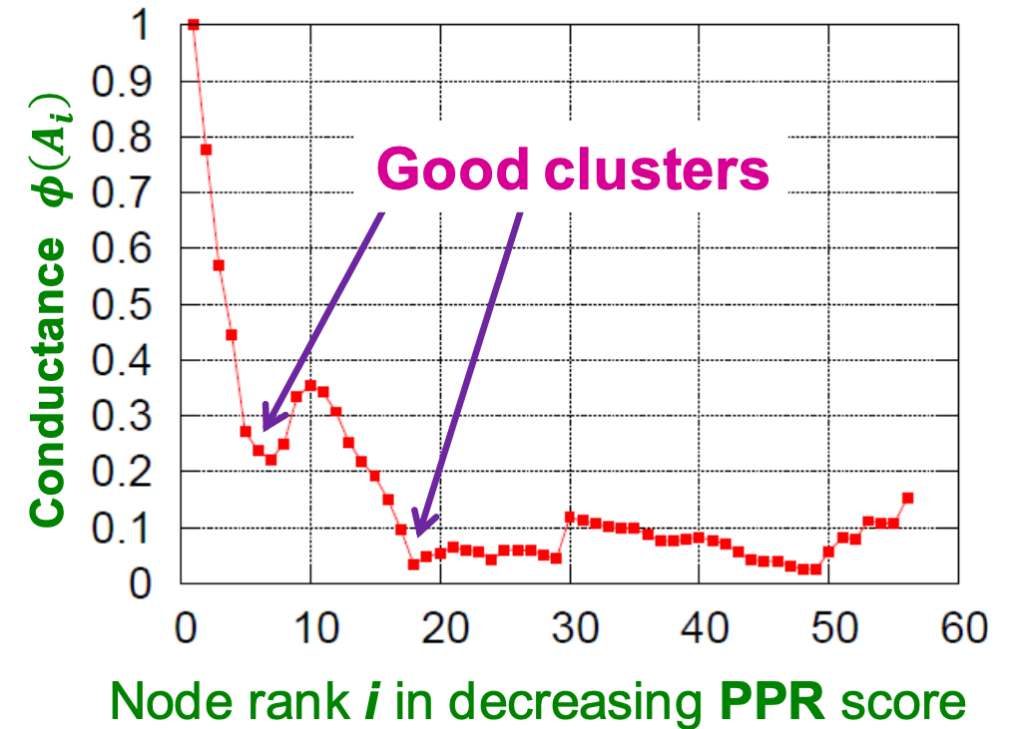
Algorithm outline: Sweep

■ Algorithm outline:

- Pick a seed node s of interest
- Run **PPR** w/ teleport= $\{s\}$
- Sort the nodes by the decreasing **PPR** score
- **Sweep** over the nodes and find good clusters

■ Sweep:

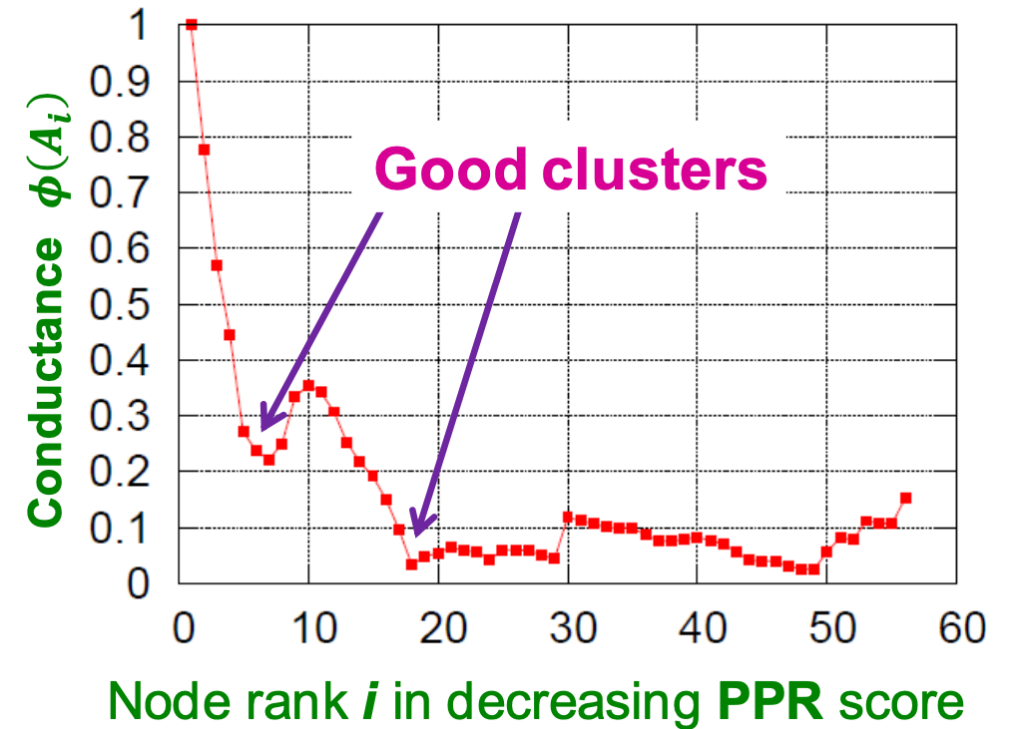
- Sort nodes by decreasing PPR score $r_1 > r_2 > \dots > r_n$
- For each i compute $\phi(A_i = \{u_1, \dots, u_i\})$
- **Local minima** of $\phi(A_i)$ correspond to good clusters



Computing the Sweep

The whole Sweep curve can be computed in **linear time**:

- For loop over the nodes
- Keep hash-table of nodes in a set A_i
- To compute $\phi(A_{i+1}) = \text{Cut}(A_{i+1}) / \text{Vol}(A_{i+1})$
 - $\text{Vol}(A_{i+1}) = \text{Vol}(A_i) + d_{i+1}$
 - $\text{Cut}(A_{i+1}) = \text{Cut}(A_i) + d_{i+1} - 2\#(\text{edges of } u_{i+1} \text{ to } A_i)$



Computing PPR

- **How to compute Personalized PageRank (PPR) without touching the whole graph?**

- Power method won't work since each single iteration accesses all nodes of the graph:

$$\mathbf{r}^{(t+1)} = \beta \mathbf{M} \cdot \mathbf{r}^{(t)} + (1 - \beta) \mathbf{a}$$

- $\mathbf{a}$ is a teleport vector: $\mathbf{a} = [0 \dots 0 \mathbf{1} 0 \dots 0]^T$
- $\mathbf{r}$ is the personalized PageRank vector

At index S



- **Approximate PageRank (AKA PageRank-Nibble)**

[Andersen, Chung, Lang, '07]

- A fast method for computing approximate Personalized PageRank (**PPR**) with teleport set $\mathbf{S} = \{\mathbf{s}\}$
- **ApproxPageRank($\mathbf{s}, \beta, \epsilon$)**
 - $\mathbf{s}$... seed node
 - β ... teleportation parameter
 - ϵ ... approximation error parameter

Approximate PPR

■ Approximate PPR on undirected graph

- **Lazy random walk**, which is a variant of a random walk that stays put with probability $\frac{1}{2}$ at each time step, and walks to a random neighbor the other half of the time:

$$r_u^{(t+1)} = \frac{1}{2} r_u^{(t)} + \frac{1}{2} \sum_{i \rightarrow u} \frac{1}{d_i} r_i^{(t)} \quad d_i \dots \text{degree of } i$$

- Keep track of **residual PPR score** $q_u = p_u - r_u^{(t)}$
 - **Residual** q_u : how well is PPR score p_u of u is approximated
 - p_u ... is the “true” PageRank of node u
 - $r_u^{(t)}$... is PageRank estimate of node u at around t

If **residual** q_u of node u is too big $q_u/d_u \geq \varepsilon$ then **push the walk further** (distribute some of residual q_u to all u 's neighbors along outgoing edges), else we don't touch the node

“Push” operation

- Idea: α ...teleport vector
 - r ... approx. PageRank, q ... its residual PageRank
 - Start with trivial approximation: $r = \mathbf{0}$ and $q = \alpha$
 - Iteratively **push** PageRank from q to r until q is small
- **Push: 1 step of a lazy random walk from node u :**

Push(u, r, q):

$$r' = r, \quad q' = q$$

$$r'_u = r_u + (1 - \beta)q_u$$

$$q'_u = \frac{1}{2}\beta q_u$$

for each v such that $u \rightarrow v$:

$$q'_v = q_v + \frac{1}{2}\beta \frac{q_u}{d_u}$$

return r', q'

$1 - \beta$...teleport prob

Update r

Do 1 step of a walk:

Stay at u with prob. $\frac{1}{2}$

Spread remaining $\frac{1}{2}$ fraction of q_u as if a single step of random walk were applied to u

Intuition behind “Push” operation

- If q_u is large, this means that we have underestimated the importance of node u
- Then we want to take some of that residual (q_u) and give it away, since we know that we have too much of it
- So, we keep $\frac{1}{2}\beta q_u$ and then give away the rest to our neighbors, so that we can get rid of it
 - This correspond to the spreading of $\frac{1}{2}\beta q_u/d_u$ term
- Each node wants to keep giving away this excess PageRank until all nodes have no or a very small gap in the excess PageRank

Push(u, r, q):

$$r' = r, \quad q' = q$$

$$r'_u = r_u + (1 - \beta)q_u$$

$$q'_u = \frac{1}{2}\beta q_u$$

for each v such that $u \rightarrow v$:

$$q'_v = q_v + \frac{1}{2}\beta \frac{q_u}{d_u}$$

return r', q'

Approximate PPR

■ **ApproxPageRank(S, β, ϵ):**

Set $\mathbf{r} = \vec{0}$, $\mathbf{q} = [0 \dots 0 \ 1 \ 0 \dots 0]$

While $\max_{u \in V} \frac{q_u}{d_u} \geq \epsilon$:  At index S

Choose any vertex u where $\frac{q_u}{d_u} \geq \epsilon$

Push($u, \mathbf{r}, \mathbf{q}$):

$$\mathbf{r}' = \mathbf{r}, \mathbf{q}' = \mathbf{q}$$

$$\mathbf{r}'_u = \mathbf{r}_u + (1 - \beta)\mathbf{q}_u$$

$$\mathbf{q}'_u = \frac{1}{2}\beta\mathbf{q}_u$$

For each v such that $u \rightarrow v$:

$$\mathbf{q}'_v = \mathbf{q}_v + \frac{1}{2}\beta\mathbf{q}_u/d_u$$

$$\mathbf{r} = \mathbf{r}', \mathbf{q} = \mathbf{q}'$$

Return $\mathbf{r}$

$\mathbf{r}$... PPR vector

r_u ... PPR score of u

$\mathbf{q}$... residual PPR vector

q_u ... residual of node u

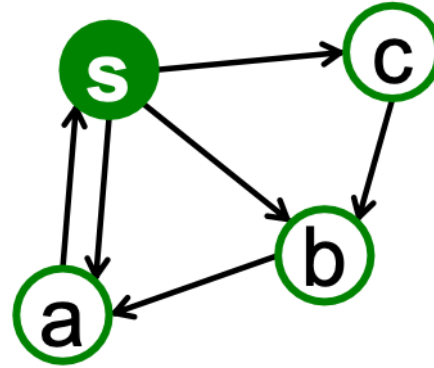
d_u ... degree of u

Update $\mathbf{r}$: Move $(1 - \beta)$
of the prob. from $\mathbf{q}_u$ to $\mathbf{r}_u$

**1 step of a lazy
random walk:**

- Stay at $\mathbf{q}_u$ with prob. $\frac{1}{2}$
- Spread remaining $\frac{1}{2}\beta$
fraction of $\mathbf{q}_u$ as if a
single step of random
walk were applied to u

Example $\beta = 0.5$



ApproxPageRank(S, β, ϵ):

Set $\mathbf{r} = \vec{0}, \mathbf{q} = [0 \dots 0 \ 1 \ 0 \dots 0]$

while $\max_{u \in V} \frac{q_u}{d_u} \geq \epsilon$:

Choose any vertex u where $\frac{q_u}{d_u} \geq \epsilon$

Push($u, \mathbf{r}, \mathbf{q}$):

$$\mathbf{r}' = \mathbf{r}, \mathbf{q}' = \mathbf{q}$$

$$\mathbf{r}'_u = \mathbf{r}_u + (1 - \beta)q_u$$

$$q'_u = \frac{1}{2}\beta q_u$$

For each v such that $(u, v) \in E$:

$$q'_v = q_v + \frac{1}{2}\beta \frac{q_u}{d_u}$$

Update $\mathbf{r} = \mathbf{r}', \mathbf{q} = \mathbf{q}'$

return $\mathbf{r}$

s a b c

Init:

$$\mathbf{r} = [0, 0, 0, 0]$$

$$\mathbf{q} = [1, 0, 0, 0]$$

Push($s, \mathbf{r}, \mathbf{q}$):

$$\mathbf{r} = [.5, 0, 0, 0]$$

$$\mathbf{q} = [.25, .08, .08, .08]$$

Push($s, \mathbf{r}, \mathbf{q}$):

$$\mathbf{r} = [.62, 0, 0, 0]$$

$$\mathbf{q} = [.06, .10, .10, .10]$$

Push($a, \mathbf{r}, \mathbf{q}$):

$$\mathbf{r} = [0.62, .05, 0, 0]$$

$$\mathbf{q} = [.09, .03, .10, .10]$$

Push($b, \mathbf{r}, \mathbf{q}$):

$$\mathbf{r} = [0.62, .05, .05, 0]$$

$$\mathbf{q} = [.09, .05, .03, .10]$$

....

$$\mathbf{r} = [0.57, 0.19, 0.14, 0.09]$$

Observations (1)

- **Runtime:**

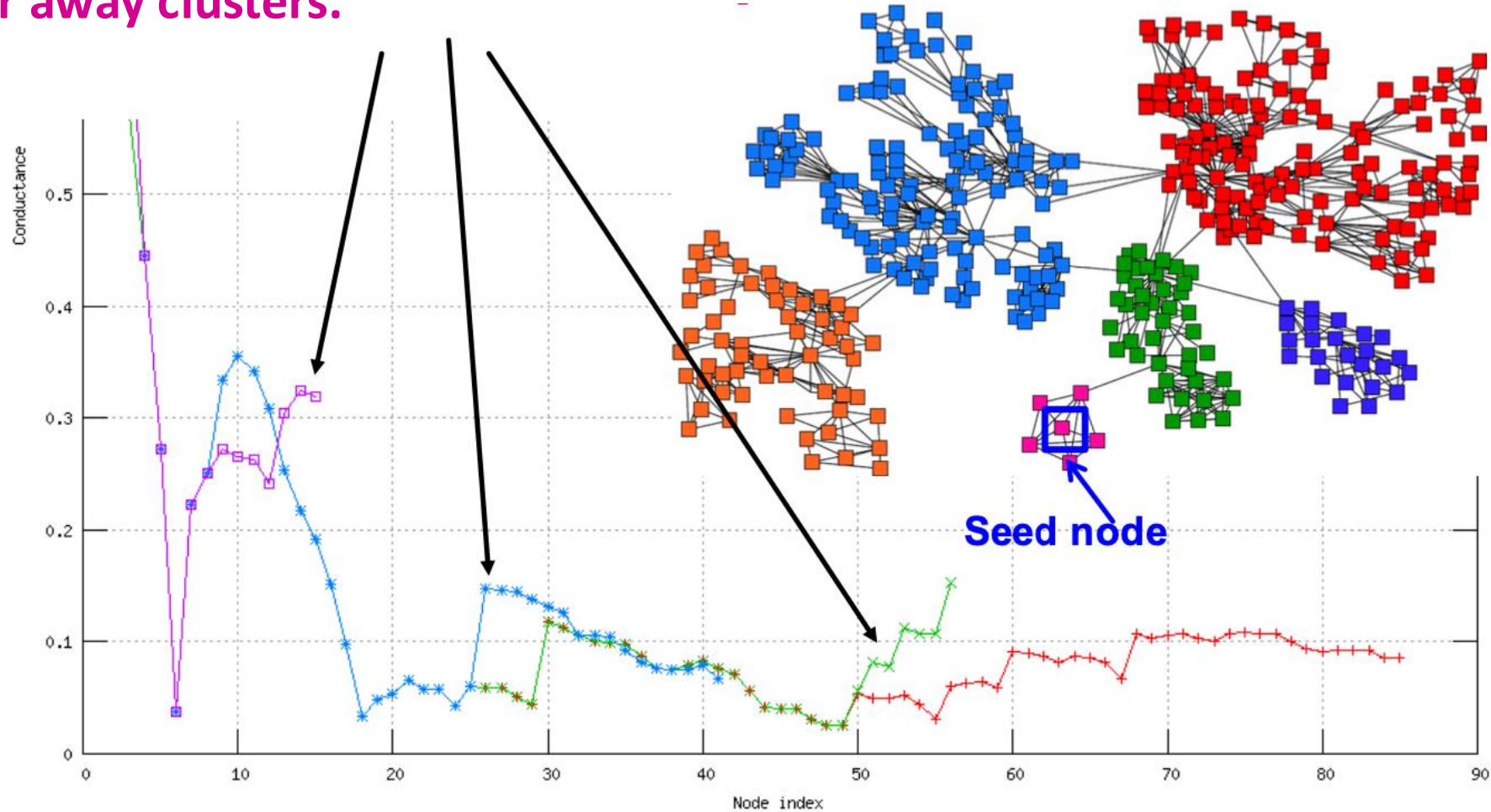
- Approximate PageRank computes PPR in time $\left(\frac{1}{\varepsilon(1-\beta)}\right)$ with residual error $\leq \varepsilon$
 - Power method would take time $\mathcal{O}\left(\frac{\log n}{\varepsilon(1-\beta)}\right)$

- **Graph cut approximation guarantee:**

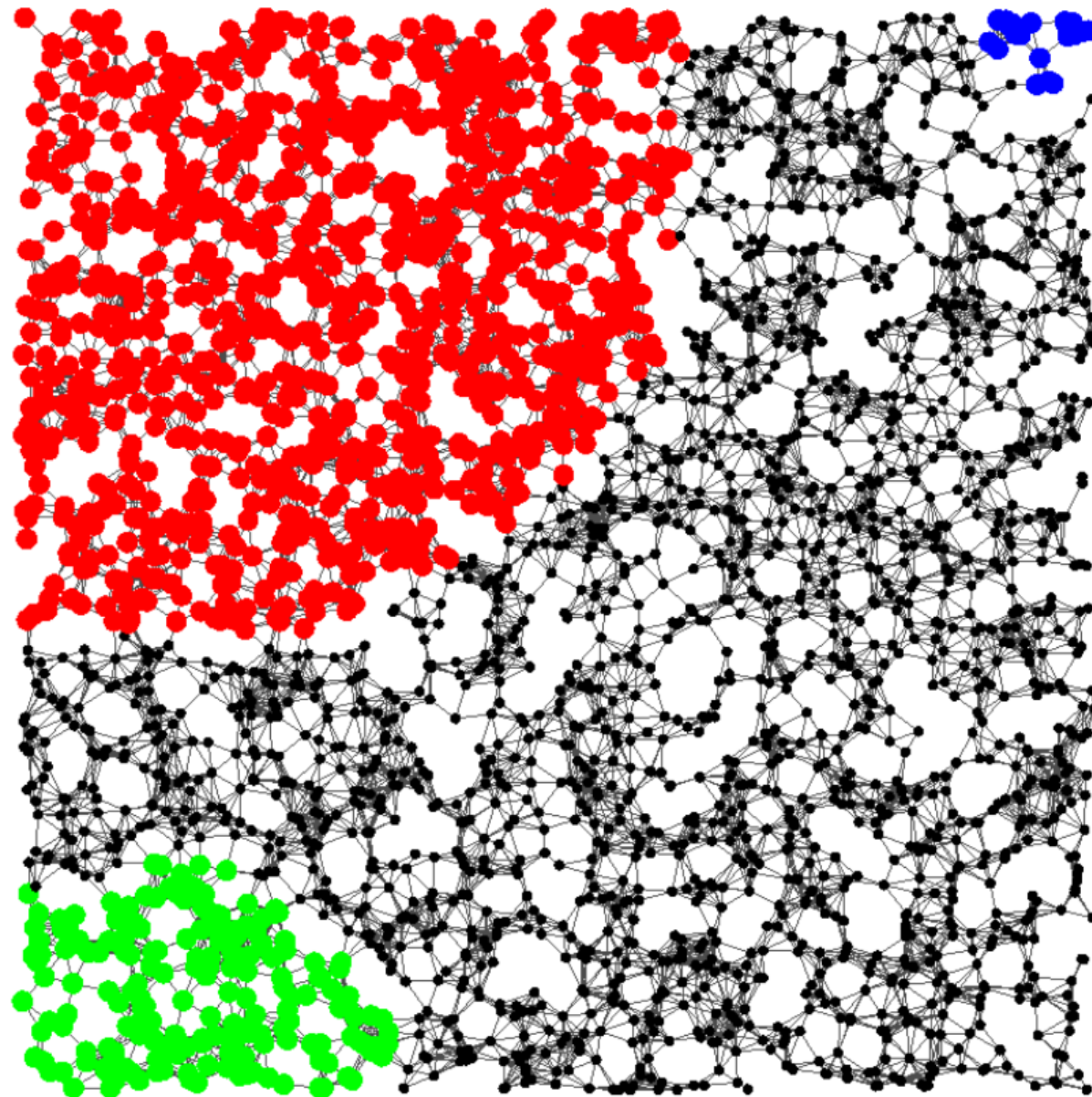
- If there exists a cut of conductance ϕ and volume k then the method finds a cut of conductance $\mathcal{O}(\sqrt{\phi / \log k})$
- Details in [Andersen, Chung, Lang. *Local graph partitioning using PageRank vectors*, 2007]

Observations (2)

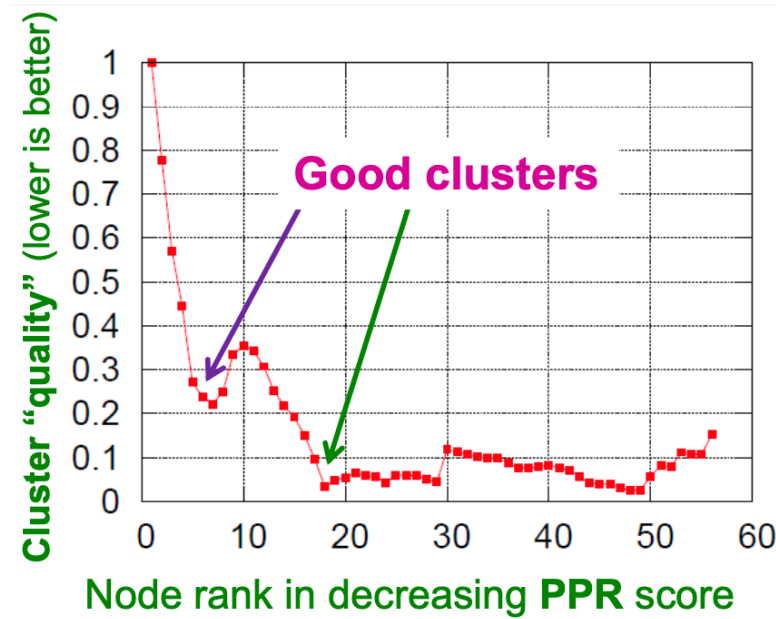
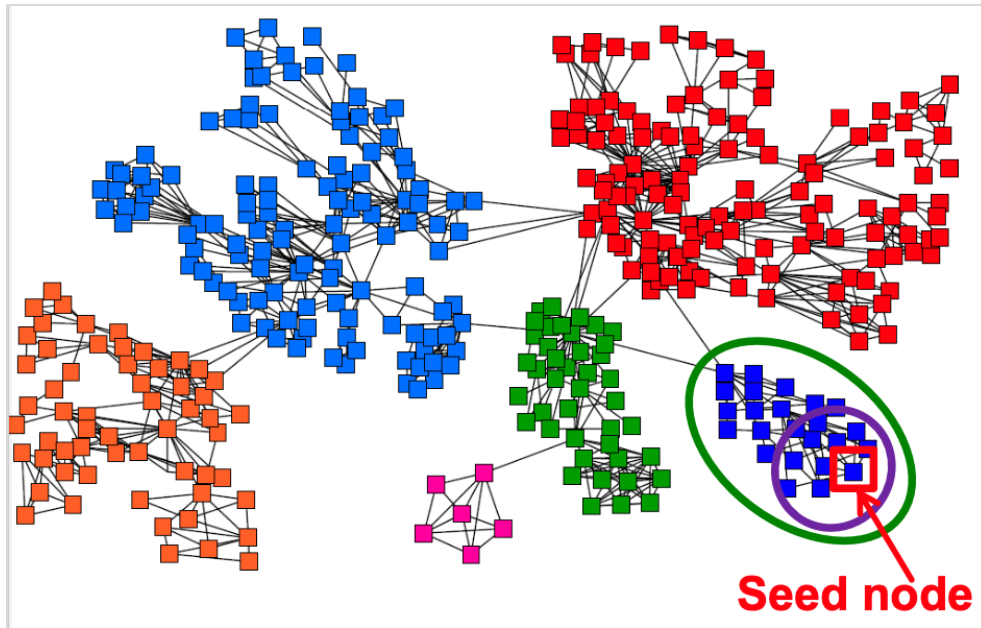
The smaller the ϵ the farther the random walk will spread and detect far away clusters.



Example



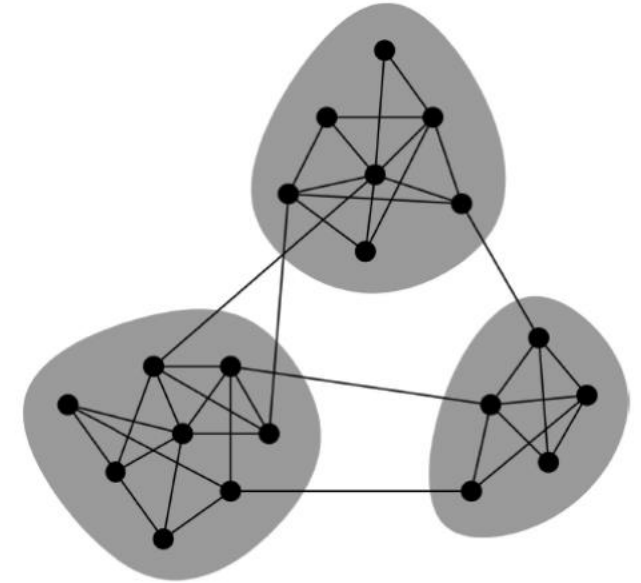
Summary of approx. PPR algorithm



■ Algorithm summary:

- Pick a seed node s of interest
- Run **PPR** with teleport set = $\{s\}$
- Sort the nodes by the decreasing **PPR score**
- **Sweep** over the nodes and find **good clusters**

Modularity Maximization



Network communities

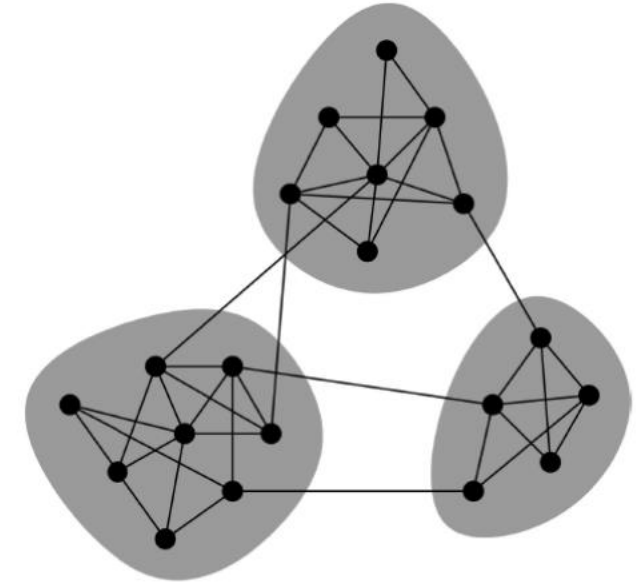
- **Communities:** sets of tightly connected nodes

- Define: **Modularity Q**

- A measure of how well a network is partitioned into communities
- Given a partitioning of the network into groups $s \in S$:

$$Q \propto \sum_{s \in S} [\underbrace{(\# \text{ edges within group } s) - (\text{expected } \# \text{ edges within group } s)}_{\text{Need a null model!}}]$$

Need a null model!



Null model: Configuration model

- Given real G on n nodes and m edges, construct rewired network G'

- Same degree distribution but random connections

- Consider G' as a **multigraph**

- The expected number of edges between nodes

i and j of degrees k_i and k_j equals to: $k_i \cdot \frac{k_j}{2m} = \frac{k_i k_j}{2m}$

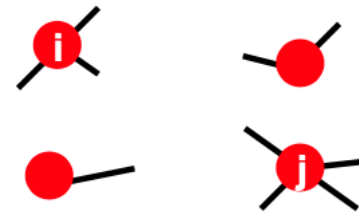
- The expected number of edges in (multigraph) G' :

- $= \frac{1}{2} \sum_{i \in N} \sum_{j \in N} \frac{k_i k_j}{2m} = \frac{1}{2} \cdot \frac{1}{2m} \sum_{i \in N} k_i (\sum_{j \in N} k_j) =$

- $= \frac{1}{4m} 2m \cdot 2m = m$

Note:

$$\sum_{u \in V} k_u = 2m$$



Modularity

- **Modularity of partitioning S of graph G:**

- $Q \propto \sum_{s \in S} [(\# \text{ edges within group } s) - (\text{expected } \# \text{ edges within group } s)]$

- $Q(G, S) = \underbrace{\frac{1}{2m}}_{\text{Normalizing const.: } -1 < Q < 1} \sum_{s \in S} \sum_{i \in s} \sum_{j \in s} \left(A_{ij} - \frac{k_i k_j}{2m} \right)$

Normalizing const.: $-1 < Q < 1$

$A_{ij} = 1$ if $i \rightarrow j$,
0 else

- **Modularity values take range $[-1, 1]$**

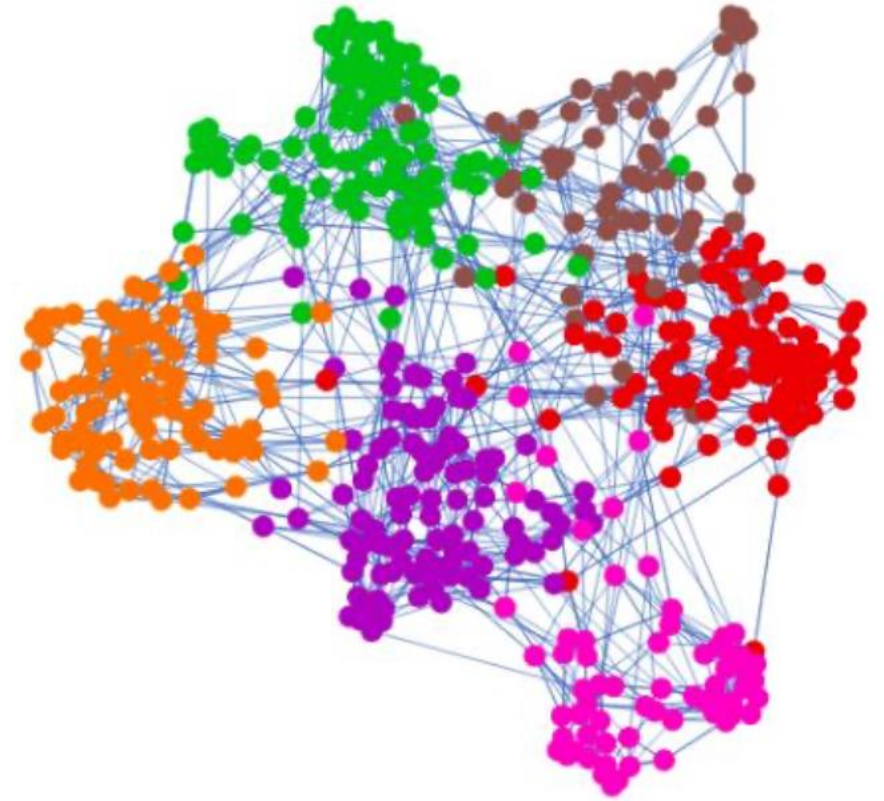
- It is positive if the number of edges within groups exceeds the expected number
- Q greater than **0.3-0.7** means **significant community structure**

Community detection

Idea: We can identify communities by maximizing modularity

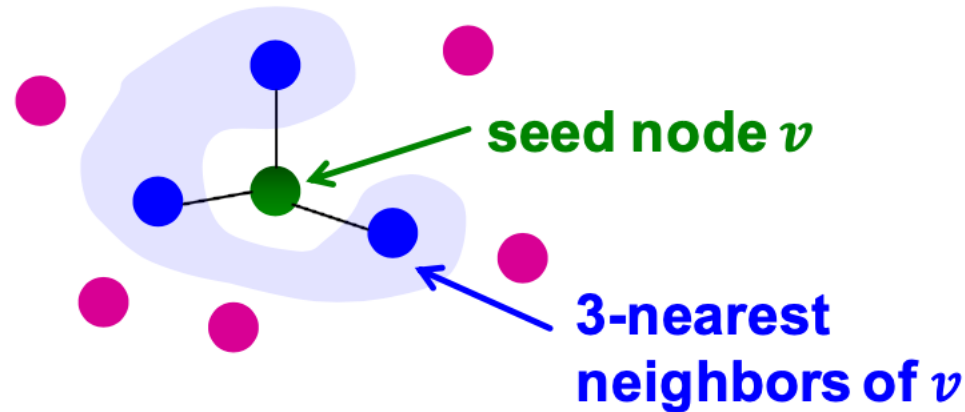
$$Q(G, S) = \frac{1}{2m} \sum_{s \in S} \sum_{i \in s} \sum_{j \in s} \left(A_{ij} - \frac{k_i k_j}{2m} \right)$$

k-NN Graph Construction



k-NN graph

- **K nearest neighbor (K-NN) graph:** Directed graph with vertex set V and an edge from each $v \in V$ to its **K most similar objects** in V under a given similarity measure
 - *E.g.*, cosine similarity, l_2 distance, l_1 distance



Compute k-NN graph

- **Brute force algorithm:**

- Takes $O(n^2)$ time
- Only practical for small datasets!

How to efficiently compute k-NN graph?

- **NN-Descent** [Dong, Charikar, Li. '11]

- Scalable method for creating approximate k-NN graph
- Suitable for large-scale datasets
- Empirical cost is around $O(n^{1.14})$
- Suitable for distributed implementation (*e.g.*, Map Reduce)

NN-Descent heuristic

NN-Descent is an iterative refinement algorithm:

- Start with a random KNN graph
 - Each node picks K random other nodes as its nearest neighbors
- Iteratively refine the list of nearest neighbors of each node:
 - **A neighbor of a neighbor could also be my neighbor**
- Keep doing this until convergence.

NN-Descent algorithm

- Start with a random K-NN list by sampling K items for every node $v \in V$
- Then iteratively for every node $v \in V$:
 - $B[v]$... is the current/approximate K-NN of v
 - $R[v]$... is the current/approximate reverse K-NN of v
 - **Reverse K-NN:** $R[v] = \{ u \in V \mid v \in B[u] \}$
 - Get **general neighbors** $B^*[v] = B[v] \cup R[v]$
 - For each **general neighbor** $u \in B^*[v]$, check the similarity between v and $B^*[u]$ (general neighbors of u are candidates for new neighbors of v)
 - Update nearest neighbors list if similarity is higher compared to the set of current approximate neighbors

Effective KNN graph construction

■ **NNDescent(V, σ, K):**

$B[v] = \text{Random sample of } K \text{ items } V, \forall v \in V$

Loop:

$R = \text{reverse}(B)$

$B^*[v] = B[v] \cup R[v], \forall v \in V$

$c = 0$

for $v \in V$:

for $u_1 \in B^*[v], u_2 \in B^*[u_1]$:

$l = \sigma(v, u_2)$

$c = c + \text{updateNN}(B[v], \langle u_2, l \rangle)$

return B **if** $c = 0$

V ... dataset

σ ...similarity oracle

K ...number of neighbors

$B[v]$... approximate neighbors of v

$R[v]$... approximate reverse neighbors of v

$B^*[v]$... approximate general neighbors of v

c ... counter

$B[v]$ is organized as a heap
→updates cost $O(\log K)$

■ **reverse(B):**

$R[v] = \{u \mid \langle v, \dots \rangle \in B[u]\}, \forall v \in V$

return R

■ **updateNN($H, \langle u, l, \dots \rangle$):**

Update KNN heap H

return 1 if changed, 0 if not

Example K=2

Neighbors:

$$B[s] = \{c, d\}$$

Reverse neighbors:

$$R[s] = \{b, c, e\}$$

General neighbors:

$$B^*[s] = \{b, c, d, e\}$$

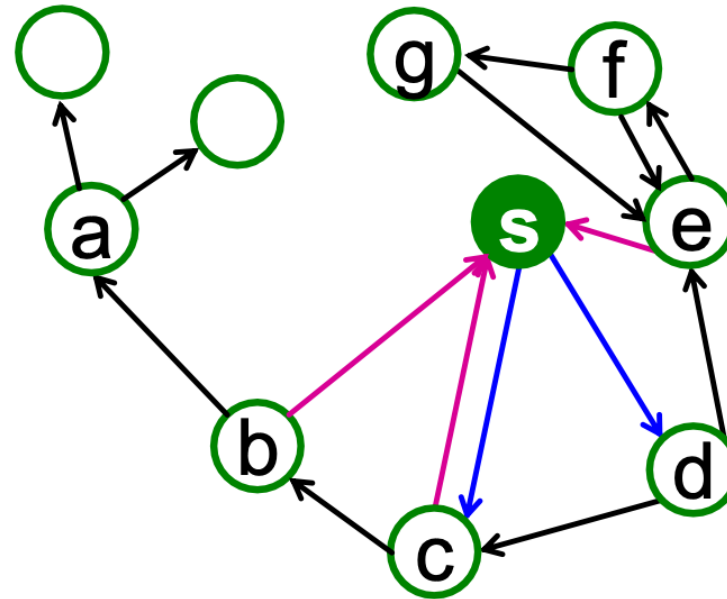
$$B^*[b] = \{a, c, s\}$$

$$B^*[c] = \{b, d, s\}$$

$$B^*[d] = \{c, e, s\}$$

$$B^*[e] = \{d, f, g, s\}$$

General neighbors of $B^*[s]$
are new candidates for $B[s]$



We will check $\{a, b, e, f, g\}$ as next candidates for $B[s]$:

Compute $\sigma(s, a)$, $\sigma(s, b)$, $\sigma(s, e)$, $\sigma(s, f)$, $\sigma(s, g)$ and update NNs of s

Arrows denote neighbors of a particular node. For example, arrow from b to s means that b selected s as its neighbor (but the opposite does not need to be true).